

Docker Interview Prep — Q&A

Prepared for Ketan Dhadve — DevOps Engineer Intern

Basic

1. What is Docker, and what problem does it solve?

Docker is a containerization platform that packages an application together with its dependencies, libraries, and runtime into a single portable unit called a container. It solves the 'works on my machine' problem — since the container includes everything the app needs, it behaves identically on a developer's laptop, a CI runner, or a production server, rather than depending on whatever happens to be installed on the host.

2. What is the difference between a Docker image and a container?

An image is a read-only, immutable template — a snapshot of an application's filesystem and configuration, built in layers from a Dockerfile. A container is a running (or stopped) instance of an image — a lightweight, isolated process with its own writable layer on top of the image. You can run many containers from the same image, each independent of the others.

3. What is a Dockerfile?

A Dockerfile is a text file containing step-by-step instructions Docker uses to build an image — specifying a base image, files to copy in, commands to run, environment variables, and the command that runs when a container starts.

4. What are the key Dockerfile instructions (FROM, RUN, COPY, CMD, ENTRYPOINT, WORKDIR, EXPOSE, ENV)?

FROM sets the base image to build on top of. RUN executes a command at build time and bakes the result into a new image layer (e.g. installing packages). COPY copies files from the build context into the image. CMD provides the default command/arguments run when a container starts (overridable at docker run). ENTRYPOINT sets the fixed executable that always runs, with CMD or run arguments appended to it. WORKDIR sets the working directory for subsequent instructions and the container's default directory. EXPOSE documents which port the container listens on (informational — doesn't actually publish the port). ENV sets an environment variable available both at build time and inside the running container.

5. What is the difference between CMD and ENTRYPOINT?

CMD specifies default arguments that are easily overridden — anything passed after the image name in docker run replaces CMD entirely. ENTRYPOINT specifies the executable that always runs; arguments passed to docker run are appended to it rather than replacing it. A common pattern combines both: ENTRYPOINT ["java", "-jar", "app.jar"] with CMD ["--spring.profiles.active=default"] as an overridable default argument.

6. What is the difference between COPY and ADD?

COPY does exactly one thing — copies files/directories from the build context into the image. ADD does that too, but also has extra 'magic' behavior: it can automatically extract local tar archives, and can fetch a file from a remote URL. Because of this hidden behavior, best practice is to always prefer COPY unless you specifically need ADD's archive-extraction feature, since ADD's implicit behavior makes builds harder to reason about.

7. What is a base image?

The base image is the starting point declared in a Dockerfile's FROM instruction — e.g. FROM eclipse-temurin:17-jre or FROM ubuntu:22.04. Every subsequent instruction builds a new layer on top of it. Choosing a minimal base image (like an -alpine or -slim variant, or a distroless image) reduces final image size and attack surface.

8. How do you build an image from a Dockerfile?

`docker build -t myapp:1.0 .` — the `-t` flag tags the resulting image, and the trailing `.` specifies the build context (the directory whose contents are sent to the Docker daemon and available to COPY/ADD instructions).

9. How do you run a container from an image?

`docker run -d -p 8080:8080 --name myapp-container myapp:1.0` — `-d` runs it detached (in the background), `-p 8080:8080` maps host port 8080 to the container's port 8080, and `--name` gives the container a readable name instead of a random one.

10. What is the difference between `docker run` and `docker start`?

`docker run` creates a brand new container from an image and starts it — every call creates a fresh container instance. `docker start` restarts an existing, previously created (but currently stopped) container by its name or ID, reusing its existing filesystem state rather than creating a new one.

11. How do you list running vs all containers?

`docker ps` lists only currently running containers. `docker ps -a` lists all containers, including stopped and exited ones.

12. How do you stop and remove a container?

`docker stop <container>` sends a graceful SIGTERM (falling back to SIGKILL after a timeout) to stop it. `docker rm <container>` removes a stopped container entirely. `docker rm -f <container>` forces removal even if it's still running (stops it first, then removes).

13. What is a Docker registry, and what's the default one (Docker Hub)?

A registry is a storage and distribution service for Docker images — you push images to it and pull them from it. Docker Hub is the default public registry Docker uses if no other registry is specified in an image name. Organizations commonly also run private registries (Amazon ECR, GitHub Container Registry, self-hosted) for internal images.

14. How do you tag and push an image to a registry?

`docker tag myapp:1.0 myregistry.com/myteam/myapp:1.0` renames/tags a local image with the target registry's path, then `docker push myregistry.com/myteam/myapp:1.0` uploads it. Docker Hub images can omit the registry hostname since it's the default.

15. What is a `.dockerignore` file used for?

It works like `.gitignore` — it excludes specified files/directories (e.g. `node_modules`, `.git`, local `.env` files) from being sent to the Docker daemon as part of the build context. This speeds up builds, avoids leaking local secrets/credentials into the image, and prevents unrelated files from invalidating the build cache.

16. What is Docker Compose?

Docker Compose is a tool for defining and running multi-container applications using a single declarative YAML file (`docker-compose.yml`). Instead of running multiple `docker run` commands by hand, you describe each service, its image/build context, ports, environment variables, volumes, and dependencies, then bring the whole stack up with `docker compose up`.

17. What is a Docker volume, and why use it over a bind mount?

A volume is storage managed entirely by Docker (stored under Docker's own directory, e.g. `/var/lib/docker/volumes`) rather than tied to a specific host path. A bind mount instead maps a specific host filesystem path directly into the container. Volumes are generally preferred for persistent application data (like a database) because they're portable across hosts, easier to back up, and not tied to the host's exact directory structure — bind mounts are better suited for live-reloading source code during local development, where you want the container to see host file changes instantly.

18. How do you view logs of a running container?

`docker logs <container>` shows accumulated stdout/stderr output. `docker logs -f <container>` follows the logs in real time (like `tail -f`).

19. How do you get an interactive shell inside a running container?

`docker exec -it <container> /bin/bash` (or `/bin/sh` if `bash` isn't available, e.g. on Alpine-based images) — `-i` keeps stdin open and `-t` allocates a pseudo-terminal, together giving an interactive shell session.

20. What is the default Docker network driver (bridge)?

bridge is the default network mode — Docker creates a private internal network on the host, and containers get their own internal IP addresses, communicating with each other and the outside world through this bridge. It's the default unless another network mode (host, none, overlay, or a custom bridge) is specified.

Intermediate / Scenario

21. What is a multi-stage Docker build, and why is it useful?

A multi-stage build uses multiple `FROM` statements in one Dockerfile, where an early stage (with a full SDK/build toolchain) compiles or builds the application, and a later, much smaller final stage copies only the compiled output from the earlier stage using `COPY --from=<stage>`. This keeps the final image free of build tools, source code, and intermediate artifacts, dramatically reducing final image size and attack surface.

22. How do you reduce the size of a Docker image?

Use a minimal base image (alpine, -slim, or distroless variants), use multi-stage builds so build tools never reach the final image, combine `RUN` commands with `&&` and clean up package manager caches in the same layer (so the cache isn't preserved in an earlier layer), use `.dockerignore` to avoid copying unnecessary files, and avoid installing unnecessary debugging/dev packages in the production image.

23. Why shouldn't containers run as root, and how do you configure a non-root user?

Running as root inside a container means that if an attacker breaks out of the application or exploits a container escape vulnerability, they have root privileges — a much larger blast radius than a restricted user. It's configured in the Dockerfile:

```
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser
```

Placed near the end of the Dockerfile so the container process runs as `appuser` rather than `root` by default.

24. What is the difference between bridge, host, and none networking modes?

bridge (default) gives the container its own private IP on a Docker-managed virtual network, isolated from the host's network stack, with ports explicitly published via `-p`. host mode removes network isolation entirely — the container shares the host's network namespace directly, so a service listening on port 8080 in the container is immediately reachable on the host's port 8080 with no port mapping needed (faster, but less isolated). none disables networking entirely — the container gets no network interface at all beyond loopback, used for fully isolated batch/processing jobs that don't need network access.

25. How do containers on the same custom network discover each other?

On a user-defined custom bridge network (created via `docker network create`), Docker provides automatic DNS resolution by container name — one container can reach another simply by using its container name or Compose service name as a hostname, without needing to know its IP address. This automatic DNS resolution does not work on the default bridge network, which is one of the main reasons to always create a custom network rather than relying on the default.

26. How do you limit a container's CPU and memory usage?

`docker run --memory=512m --cpus=1.5 myapp:1.0` — `--memory` caps RAM usage (the container is OOM-killed if it exceeds this), and `--cpus` limits how much CPU time it can use. In Compose, the equivalent is set under `deploy.resources.limits`.

27. What is a Docker health check (HEALTHCHECK), and why is it useful?

HEALTHCHECK defines a command Docker runs periodically inside the container to determine if the application is actually healthy, not just that the process is running:

```
HEALTHCHECK --interval=30s --timeout=3s CMD curl -f http://localhost:8080/health || exit 1
```

This lets Docker (and orchestrators built on top of it) distinguish between a container that's merely 'up' and one that's genuinely ready to serve traffic — useful for rolling deployments and automatic restarts of containers that are running but unresponsive.

28. How do you persist database data across container restarts?

Mount a named volume at the database's data directory, e.g. for MySQL: `docker run -v mysql_data:/var/lib/mysql mysql:8`. Since the volume is managed independently of the container's lifecycle, removing and recreating the container (e.g. during an image upgrade) doesn't lose the data — only explicitly removing the volume itself does.

29. What is image layering, and how does it affect build caching?

Each instruction in a Dockerfile creates a new filesystem layer stacked on top of the previous one, and Docker caches each layer. On a rebuild, Docker reuses cached layers for any instruction whose inputs haven't changed, only re-executing from the first changed instruction onward. This is why instruction order matters: placing rarely-changing steps (like installing dependencies) before frequently-changing steps (like copying application source code) maximizes cache reuse and speeds up rebuilds significantly.

30. How would you design a Dockerfile for a Java Spring Boot app using multi-stage builds?

A build stage using a full JDK + Maven image compiles and packages the application, then a final stage using a minimal JRE-only base image copies just the built `.jar`:

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Copying `pom.xml` before the source code and running `dependency:go-offline` first lets Docker cache the downloaded Maven dependencies as a separate layer, so changing application source code alone doesn't force re-downloading all dependencies on every build — directly applicable to your `FlightReservationApplication` and `FlightCheckInApplication` Spring Boot services.

31. What is the difference between `docker exec` and `docker attach`?

`docker exec` starts a brand new process inside an already-running container (e.g. opening a new shell alongside the main application process) — the most common way to inspect or debug a running container. `docker attach` instead connects your terminal directly to the container's own main process (PID 1) and its existing `stdin/stdout/stderr` streams — useful for interacting with the container's actual foreground process, but risky since pressing `Ctrl+C` can send a signal that stops the container's main process entirely.

32. How do you pass environment variables and secrets into a container securely?

Plain configuration can be passed with `-e KEY=value` or an `--env-file`. For actual secrets, avoid baking them into the image or passing them as visible build arguments (which end up in image history/layers); instead use Docker secrets (in Swarm), a mounted file/volume containing the secret that the app reads at startup, or — in a cloud context — fetch the secret at container startup from a secrets manager (e.g. AWS Secrets Manager) using the container's IAM role, so the value never lives in an environment variable visible via `docker inspect` or in the image itself.

33. What is Docker's overlay network used for (multi-host networking)?

An overlay network lets containers running on different physical/virtual hosts communicate with each other as if they were on the same local network, by encapsulating traffic between hosts. It's used in multi-host orchestration setups like Docker Swarm, where a service's containers may be scheduled across several nodes but still need to discover and reach each other transparently.

34. How would you scan a Docker image for vulnerabilities before pushing to production?

Run a vulnerability scanner like Trivy against the built image before pushing, e.g. `trivy image myapp:1.0`, which checks OS packages and application dependencies against known CVE databases and reports severity levels. This is commonly wired into a CI/CD pipeline as a gate — failing the build if critical/high vulnerabilities are found — which aligns with the Trivy and OWASP Dependency-Check steps already used in your Jenkins pipeline for the Flight Reservation project.

35. What's the difference between an image's latest tag and a versioned tag, and why avoid latest in production?

`latest` is just a regular, mutable tag by convention — whoever pushes an image without specifying a tag gets `latest` by default, meaning the image it points to can silently change over time as new versions are pushed. A versioned tag (e.g. `myapp:1.4.2` or a Git commit SHA) points to one specific, immutable build. Using `latest` in production is risky because a redeploy or scale-out event could pull a completely different (and untested) image version than what was originally deployed, making rollbacks and reproducibility unreliable.

36. How do you troubleshoot a container that exits immediately after starting?

Run `docker logs <container>` first to see the exit output/error. If logs are empty, check `docker inspect <container>` for the exit code (e.g. 0 = clean exit because the main process finished immediately, 1 = general application error, 137 = killed, often OOM). A frequent cause is `CMD/ENTRYPOINT` running a process that isn't actually long-running (e.g. a script that finishes instead of a server that blocks) — Docker containers exit as soon as their main process (PID 1) exits.

37. What is the difference between `docker stop` and `docker kill`?

`docker stop` sends `SIGTERM` first, giving the application a grace period (default 10 seconds) to shut down cleanly, then sends `SIGKILL` if it hasn't exited by the timeout. `docker kill` sends `SIGKILL` immediately, terminating the container abruptly with no chance for graceful cleanup — useful when a container is unresponsive and `stop` would otherwise hang until the timeout.

38. How do you copy files between your host machine and a running container?

`docker cp ./localfile.txt <container>:/path/in/container` copies from host to container, and `docker cp <container>:/path/in/container ./localfile.txt` copies from container to host — works on running or stopped containers, without needing a shared volume.

39. What is Docker BuildKit, and what benefits does it provide?

BuildKit is Docker's modern build engine (default in current Docker versions) that replaces the legacy builder. It provides parallel execution of independent build stages, better layer caching (including cache mount support for package manager caches across builds), the ability to skip unused build stages entirely, secret mounting during build (`RUN --mount=type=secret`) without leaking secrets into image layers, and generally faster, more efficient builds than the legacy builder.

40. How would you set up a local development environment with Docker Compose for a multi-service app (frontend, backend, database)?

Define each service as its own entry in `docker-compose.yml`, each with its own build context or image, environment variables, and port mappings, using `depends_on` to express startup ordering and a shared custom network (Compose creates one automatically) so services can reach each other by service name:

```
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    depends_on:
      - db
    environment:
      - DB_HOST=db
  db:
    image: mysql:8
    environment:
      - MYSQL_ROOT_PASSWORD=example
    volumes:
      - db_data:/var/lib/mysql
volumes:
  db_data:
```

The backend reaches the database simply via the hostname `db` (the service name), and bind-mounting local source directories into frontend/backend during development enables live-reload without rebuilding the image on every change.

Troubleshooting

41. A container keeps restarting in a crash loop. How do you investigate the root cause?

Check docker logs `<container>` for the application error right before each crash, and `docker inspect <container>` for the exit code and current restart policy. If logs cut off with no clear error, the process may be getting OOM-killed (exit code 137) — check `docker stats` or resource limits. Also verify the restart policy itself (`--restart=always` or `unless-stopped`) isn't masking a real crash by continuously restarting instead of surfacing the failure clearly.

42. An image builds successfully but the container fails at runtime with "file not found." What's a likely cause?

A very common cause is a mismatched `WORKDIR` or relative path — a `COPY` instruction placed a file at one path during build, but the `CMD/ENTRYPOINT` or application code references a different path at runtime. Another common cause in multi-stage builds is forgetting to `COPY --from=build` the compiled artifact into the final stage at all, so the final image never actually contains the file the entrypoint expects.

43. Two containers on the same custom bridge network can't reach each other by name. What would you check?

Confirm both containers are actually attached to the same custom network (not the default bridge, which lacks automatic DNS) via `docker network inspect <network>`. Check for typos in the hostname used (it must match the container name or Compose service name exactly), and confirm the target container's application is actually listening on `0.0.0.0` rather than only `127.0.0.1` inside the container — binding to `localhost` only accepts connections from within that same container, not from other containers on the network.

44. A container is consuming excessive memory and eventually gets OOM-killed. How do you diagnose and fix it?

Use `docker stats` to observe real-time memory usage, and `docker inspect` for the configured `--memory` limit and the `OOMKilled` flag confirming it was killed for exceeding it. Diagnose whether the application itself has a memory leak (check JVM heap settings for a Java app, for instance — a container's memory limit needs to account for `-Xmx` plus JVM overhead, not just the heap) or whether the limit set was simply too low for legitimate usage, then either fix the underlying leak or adjust `--memory` to a realistic value.

45. `docker build` is extremely slow due to cache invalidation on every build. How would you restructure the `Dockerfile` to fix this?

Order instructions from least-frequently-changing to most-frequently-changing: base image and OS-level dependency installation first, then language/package-manager dependency files (`package.json`, `pom.xml`, `requirements.txt`) copied and installed separately, and only then copy the actual application source code last. Since Docker invalidates the cache from the first changed layer onward, this ensures that changing only application code (the common case) doesn't force reinstalling all dependencies from scratch on every build.

46. An application works when run manually inside the container but fails when started via `CMD/ENTRYPOINT`. What could differ (working directory, env vars, user)?

Manually running a shell inside the container often starts in a different working directory than `WORKDIR` sets for the entrypoint, so relative paths that worked interactively may not resolve the same way. Environment variables set only in your interactive shell session (or inherited from a login profile) aren't automatically present for the process Docker starts via `CMD/ENTRYPOINT` unless declared with `ENV` or passed via `-e`. Finally, an interactive docker `exec` shell typically runs as whatever user was specified, but if `USER` differs between manual testing and the actual entrypoint process, file permission issues can appear only in one case.

47. A pushed image works on your amd64 laptop but fails with "exec format error" on a production ARM server. What's the issue and fix?

The image was built for one CPU architecture (`amd64/x86_64`) but is being run on a different one (`ARM/aarch64`) — container images are architecture-specific unless built as multi-arch. The fix is to build a multi-platform image using Docker Buildx, e.g. `docker buildx build --platform linux/amd64,linux/arm64 -t myapp:1.0 --push .`, which produces and pushes a single manifest that automatically serves the correct architecture-specific image to whichever platform pulls it.

48. Disk space on a Docker host is filling up unexpectedly. What Docker resources would you check and clean up (dangling images, stopped containers, volumes)?

`docker system df` shows a breakdown of space used by images, containers, volumes, and build cache. `docker image prune` removes dangling (untagged) images left over from repeated builds; adding `-a` removes all images not currently used by any container. `docker container prune` removes stopped containers, `docker volume prune` removes volumes not attached to any container, and `docker builder prune` clears old BuildKit cache. `docker system prune -a -volumes` is the aggressive all-in-one cleanup, but should be used carefully since it also removes unused volumes that might still hold needed data.

49. A container can't resolve external DNS names, though the host machine can. What would you check?

Check the container's `/etc/resolv.conf` (via `docker exec`) to see what DNS servers it's actually using — Docker normally copies the host's DNS config, but corporate VPNs or certain host DNS setups (like `systemd-resolved`'s local stub resolver at `127.0.0.53`) don't translate correctly into the container's network namespace, since the container can't reach the host's loopback-only resolver. The fix is typically specifying explicit DNS servers for the container or daemon, e.g. `docker run --dns=8.8.8.8 ...` or setting the `dns` key in Docker's `daemon.json` / `Compose` file.

50. Docker Compose brings up services in the wrong order, causing app startup failures. How do you fix dependency ordering?

`depends_on` in Compose only controls the order containers are started, not whether the dependency is actually ready to accept connections yet (e.g. a database container can be 'running' long before MySQL has finished initializing).

The fix is to combine `depends_on` with a `HEALTHCHECK` on the dependency and `depends_on`: `condition: service_healthy` on the dependent service, so Compose waits for the database to report itself as actually healthy before starting the app that needs it — rather than just waiting for the container process to start.